What Do Spikes Buy in Flow-Based Intrusion Detection? A Leakage-Controlled Study of an Ensemble Spiking Network on CSE-CIC-IDS2018

Özgür Uçar
ucar45ozgur@gmail.com

***Abstract*—Spiking neural networks (SNNs) are often proposed for network intrusion detection because event-driven hardware could run them cheaply at the network edge. Most reported results, however, come from random row splits of benchmark datasets that contain many exact duplicates, and from comparisons that leave out strong tree ensembles. We revisit the question on CSE-CIC-IDS2018 (16.2 million flows, 13 classes) with a protocol built to avoid those problems. Rows that share an identical cleaned feature vector are kept in the same partition. Model fitting, probability calibration, alarm-threshold selection and testing use four disjoint partitions, and every model is judged at the same benign false-alarm budget. We compare an ensemble of three SNN branches (rate, latency and population coding, 86k parameters) against a parameter-matched multilayer perceptron, a control network that receives the same spike-encoded input without spiking dynamics, and three tree ensembles. At a 5% false-alarm budget the SNN reaches a macro-F1 of 0.769 ± 0.022 over three seeds. This is above the matched perceptron (0.756) but below the spike-input control (0.787), random forest (0.788), CatBoost (0.791) and XGBoost (0.808). Attack detection rates differ by less than one point across all models. A feature-group audit shows that most of the remaining error is not a modelling problem: 99.99% of FTP-BruteForce flows and nearly all DoS-SlowHTTPTest flows have feature vectors that also occur under another label. Shortening the SNN's time window from 12 to 4 steps leaves macro-F1 unchanged (0.766 ± 0.015) while cutting synaptic events per flow by a factor of 2.9. Whether that makes the SNN cheaper than a conventional network depends on the cost model. With optimistic 45 nm accumulate costs the 4-step SNN needs 69% less synaptic energy than the matched perceptron, but with published Intel Loihi per-operation costs it needs about 3.0 μJ per flow against 0.4 μJ for the perceptron on a digital accelerator. A screen of five architectural variants, ranked on the tuning partition only, found no variant that clearly beats the original. The spike-input control remains both more accurate and cheaper at every window length, so our reading is that the input encoders, not the spiking dynamics, carry most of the benefit in this setting. Finally, the frozen models do not transfer to CIC-IDS2017: at a recalibrated 5% false-alarm rate they detect only 23%–32% of attack flows. About 1,100 labelled local flows are enough to bring both the SNN and XGBoost back above 98% detection, although the SNN still misses botnet traffic. We release code, frozen checkpoints and a containerised detector.**

***Index Terms*—intrusion detection, spiking neural networks, CSE-CIC-IDS2018, data leakage, calibration, neuromorphic computing**

I. Introduction

Network intrusion detection systems (NIDS) that classify flow records are attractive because flow export is cheap and already deployed in most networks. Learning-based detectors trained on public benchmarks such as CSE-CIC-IDS2018 [1] routinely report accuracies above 99%. Spiking neural networks [2] have joined this line of work, motivated by the low power that neuromorphic chips such as Loihi [3] promise for always-on monitoring.

Two problems make many of these numbers hard to interpret. The first is the data. The CIC datasets contain labelling and feature-extraction errors [4], [5], and a large share of their rows are exact duplicates. When a random row split puts copies of the same vector into both training and test sets, the test score partly measures memorisation [6], [7]. The second is the comparison. SNN papers often compare against a convolutional or recurrent network trained in a similar way, while gradient-boosted trees, which remain very hard to beat on tabular data [8], are left out. Reporting accuracy at the default argmax decision also hides the operating point that matters to an analyst: how many attacks are caught at a fixed rate of false alarms.

This paper asks a narrow question. If the evaluation controls duplicate leakage, calibrates every model the same way and fixes the false-alarm budget, does an ensemble SNN offer an accuracy or efficiency advantage on CSE-CIC-IDS2018? Our contributions are:

- A leakage-controlled protocol for CSE-CIC-IDS2018 in which identical cleaned feature vectors never cross partitions, with separate partitions for model selection, probability calibration, alarm calibration and test.
- A comparison of a three-branch ensemble SNN against a parameter-matched perceptron, a *spike-input control* that isolates the encoders from the spiking dynamics, and three tree ensembles, over three seeds and three false-alarm budgets.
- An audit of duplicated feature vectors that bounds the error any model can reach on this representation and explains the two classes on which every model fails.

- A time-window ablation ($T \in \{4, 8, 12\}$) on the same partitions with operation counts, and a screen of five architectural variants selected on the tuning partition, with every test result disclosed.
- A containerised detector that serves the frozen checkpoints for testing on new flow records.

The headline result is mixed and, we think, useful. The SNN is a competent detector and can run with a third of its original time window at no cost in accuracy. In this setting, however, it neither matches the tree ensembles nor beats a non-spiking network that receives the same encoded input.

II. Related Work

Dataset quality. Engelen et al. [4] documented flow-construction and labelling errors in CICIDS2017, and Liu et al. [5] extended the analysis to CSE-CIC-IDS2018. Arp et al. [7] list sampling bias and data snooping among the most common pitfalls of learning-based security research, and Sommer and Paxson [9] warned early that closed-world benchmark success rarely transfers to operational networks. We follow these recommendations in the split design and in how we report the remaining errors.

Spiking networks. Surrogate-gradient training [10] made it practical to train SNNs with backpropagation through time, and libraries such as snnTorch [11] brought this into standard deep-learning tooling. Tabular inputs must first be turned into spike trains. Rate, latency and population codes are the usual choices, and the choice of code can matter as much as the neuron model. Energy claims for SNNs normally rest on the fact that an accumulate costs less than a multiply-accumulate [12], which only pays off when spikes are sparse and time windows are short.

SNN-based intrusion detection. Zhou and Li [13] trained non-leaky, single-spike temporal-coded networks on NSL-KDD and AWID and reported higher accuracy than a range of deep and classical learners, although their tree-ensemble figures were taken from earlier work rather than rerun. Most later studies train leaky integrate-and-fire networks with surrogate gradients. Wang et al. [14] combined convolutional and spiking layers on CSE-CIC-IDS2018 and CIC-DDoS2019, matching an optimised CNN with far fewer parameters, and compared only against neural models. Živadinović and Simić [15] trained a small spiking classifier on BoT-IoT samples, and Fesu et al. [16] set an SNN against a CNN on NSL-KDD. Karthik et al. [17] reported detection at a 1% false-alarm rate and cross-dataset scores for a transformer–spiking hybrid, but on random percentage splits. Closest to our concerns, Patel et al. [18] benchmarked 27 neuron and encoding configurations under capture- and scenario-aware splits and found latency coding to matter more than the neuron model; their comparison stays within spiking models. Hardware evidence is thin. Alom and Taha [19] ran a detector on a physical TrueNorth chip, and Mehrabi and van Schaik [20] built a packet-level SNN on an FPGA. What we add is a combination. We control duplicate leakage on CSE-CIC-IDS2018 and judge every model, tree ensembles and a spike-input control included, at fixed false-alarm budgets. The frozen detectors are then tested on CIC-IDS2017.

TABLE I
Support per class after the feature-group split. Only training is capped.

Class	Train used	Tune	Cal.	Test
Benign	335,515	1,362,310	1,330,511	2,699,900
Bot	16,651	22,981	33,237	54,821
DDoS-HOIC	26,260	68,256	69,218	134,836
DDoS-LOIC-UDP	1,066	159	181	324
DDoS-LOIC-HTTP	23,516	57,929	57,319	115,366
DoS-GoldenEye	10,601	4,135	4,078	8,362
DoS-Hulk	20,762	46,577	45,539	92,601
DoS-SlowHTTPTest	14,486	6,487	568	11,441
DoS-Slowloris	6,422	1,053	1,064	2,451
FTP-BruteForce	16,781	1,879	6,095	7,049
Infiltration	13,497	16,205	16,432	32,470
SSH-Bruteforce	13,887	9,435	9,888	61,742
Web Attacks	556	87	106	179

Calibration and operating points. Class-prior correction [21] and temperature scaling [22] are standard ways to fix probability estimates after training on a resampled set. Neyman–Pearson classification [23] frames detection as maximising power under a constraint on the false-positive rate. That is the framing we use for alarm thresholds.

III. Data and Protocol

A. Dataset and cleaning

We use the ten daily CSV files of CSE-CIC-IDS2018 as distributed on Kaggle. After removing repeated header lines and rows with unparseable timestamps, 16,232,929 flows remain. Original labels are mapped to 13 classes; the three web-attack labels (brute force, XSS and SQL injection) are merged into one *Web Attacks* class because together they have only 928 flows. Flow identifiers, IP addresses, source port and timestamp are dropped before learning. Non-finite values are replaced by zero, and we record how often this happens for each column.

B. Feature-group split

We compute a 128-bit fingerprint of each row's cleaned feature vector and treat all rows with the same fingerprint as one group. Groups, not rows, are assigned to the training, tuning, calibration and test partitions in a 60/10/10/20 ratio. A group is not a network session; it only guarantees that no test vector was seen verbatim during training. Classes with many duplicates shrink a great deal under this rule. DDoS-HOIC, for example, has 686,012 flows but only 198,861 distinct vectors. Table I gives the resulting support.

The training partition is capped at 500,000 rows. Benign rows are subsampled, and the rarest classes are kept in full. No synthetic oversampling is used. The calibration partition is split again into a part used to fit probability calibration and a part used only to choose alarm thresholds.

C. Preprocessing

Every preprocessing step is fitted on the training partition only. Constant columns are removed, columns with absolute skew above 5 get a signed $\log(1 + |x|)$ transform, and a quantile transformer maps each feature to $[0, 1]$. Seventy features remain.

D. Decision rule

All models output 13 class scores, which we treat as logits z (tree ensembles use log-probabilities). Because training was class-capped, we first add a prior offset $\log \pi_c^{\text{full}} - \log \pi_c^{\text{train}}$ [21]. A single temperature T_{cal} is then fitted by minimising negative log-likelihood on the probability-calibration rows [22]. The alarm score is the attack log-odds

$$s(x) = \log \sum_{c \neq \text{Benign}} e^{z_c / T_{\text{cal}}} - z_{\text{Benign}} / T_{\text{cal}}. \tag{1}$$

For a false-alarm budget τ, the threshold t_τ is the empirical $(1 - \tau)$ quantile of s over benign rows of the alarm-calibration partition. A flow raises an alarm if $s(x) > t_\tau$ and is then assigned the most probable attack class. We use $\tau \in \{1\%, 5\%, 10\%\}$ and fix $\tau = 5\%$ as the primary budget before testing. Test labels are never used to pick a threshold, a temperature or a checkpoint.

IV. Models

Input. Every model receives one static vector of 70 flow statistics per flow, as exported by CICFlowMeter (durations, packet and byte counts, inter-arrival statistics, flag counts and similar). None of the models sees packet sequences. The time steps of the SNN are therefore an encoding device that turns each static value into a short spike train; they do not correspond to time in the network trace.

Ensemble SNN (E-SNN). Three branches share the same architecture and differ only in how the 70 features become spikes over $T = 12$ time steps. The *rate* branch uses a deterministic phase-accumulator code. The *temporal* branch emits a single spike whose latency decreases with feature value. The *population* branch expands each feature into four Gaussian receptive fields and rate-codes their activations. The three codes suit different features. Rate coding preserves magnitude for smooth, wide-range statistics such as byte rates, latency coding gives the largest values the earliest spikes, and population coding separates the narrow bands that many protocol-specific features such as flag counts and window sizes take. Each branch has two hidden layers of 128 and 64 leaky integrate-and-fire neurons with learnable decay, layer normalisation and dropout of 0.2, followed by a linear readout averaged over time. The three branch outputs are combined as 0.6 times a softmax-weighted average plus 0.4 times a small meta-network. This mixture was fixed in an earlier version of the study and not tuned here; the architecture screen below includes the pure weighted average $(1.0/0)$ as one variant. The model has 86,013 parameters and is trained with surrogate gradients (fast sigmoid, slope 25) using snnTorch [11], for at most 35 epochs with early stopping on tuning macro-F1.

TABLE II
TEST RESULTS AT THE 5% BENIGN FALSE-ALARM BUDGET (MEAN ± S.D., THREE SEEDS). ADR: ATTACK DETECTION RATE.

Model	Macro-F1	ADR	FPR
E-SNN	0.769 ± 0.022	0.959 ± 0.001	0.049
Matched ANN	0.756 ± 0.004	0.958 ± 0.000	0.050
Enc-ANN (control)	0.787 ± 0.017	0.963 ± 0.001	0.049
Random forest	0.788 ± 0.001	0.965 ± 0.000	0.050
CatBoost	0.791 ± 0.005	0.965 ± 0.000	0.049
XGBoost	$\mathbf{0.808 \pm 0.005}$	$\mathbf{0.967 \pm 0.000}$	0.049

Matched ANN. A two-hidden-layer perceptron whose width is chosen by parameter count alone to match the E-SNN (85,945 parameters). It sees the quantile-scaled features directly.

Spike-input control (Enc-ANN). The same three fixed encoders and fusion head as the E-SNN, but each branch flattens the whole spike train and passes it through an ordinary feed-forward network of matched size (84,503 parameters). Comparing E-SNN with Enc-ANN separates the effect of the encoding from the effect of membrane dynamics.

Tree ensembles. Random forest [24], XGBoost [25] and CatBoost [26] with fixed, commonly used hyperparameters. We did not tune them, which if anything favours the neural models.

Every neural and tree model is trained with plain cross-entropy for three seeds (42, 43, 44). As an additional check, the E-SNN and matched ANN are also trained with class-weighted cross-entropy.

Two follow-up experiments reuse the prepared partitions, preprocessor and decision rule byte for byte; a derived experiment may change only fields that act after preprocessing, and the code refuses to run if the parent's data hashes or package versions differ. The first retrains the E-SNN and Enc-ANN with $T = 4$ and $T = 8$ for all three seeds. The second trains five E-SNN variants at $T = 8$ with seed 42 and ranks them by tuning-partition macro-F1; the winner is then retrained with all three seeds.

V. Results

A. Detection at a fixed false-alarm budget

Table II reports test results at the primary 5% budget. The achieved test false-alarm rate is between 4.9% and 5.0% for every model, so the budget transfers well from calibration to test. The Kolmogorov–Smirnov distance between benign calibration and benign test scores is 0.006 for the E-SNN.

All six models catch about 96% of attack flows. Most of the separation shows up in macro-F1, which weights the rare and difficult classes equally. The E-SNN is 1.3 points above the matched ANN, which suggests the spike encoders help. It is 1.8 points below the spike-input control, though, and 3.9 points below XGBoost. Its seed variance is also the largest of all models: one of the three seeds reached 0.744. The ranking holds at the other budgets. At $\tau = 1\%$, for instance, XGBoost reaches 0.848 and the E-SNN 0.789, with

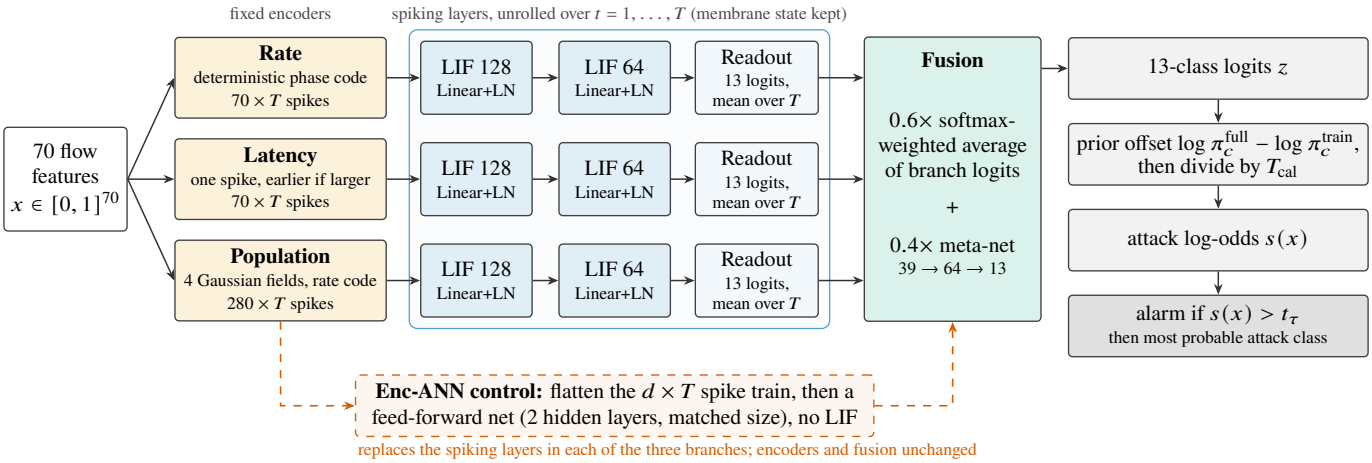


Fig. 1. Ensemble SNN. Three fixed encoders turn the 70 scaled flow features into spike trains over T steps, each branch passes them through two LIF layers and a time-averaged readout, and the branch logits are fused before the prior offset, temperature and alarm threshold t_τ are applied. The dashed path is the spike-input control (Enc-ANN), which replaces the spiking layers with a feed-forward network on the flattened spike train.

TABLE III
EFFECT OF THE TIME WINDOW. MACRO-F1 ON TEST (MEAN ± S.D., THREE SEEDS); OPERATIONS PER FLOW FOR THE SEED-42 CHECKPOINT; ENERGY WITH THE OPTIMISTIC 45 NM MODEL OF TABLE VI (SEE TABLE VII FOR LOIHI COSTS).

Model	T	MF1 @5%	MF1 @1%	ACs	Energy
E-SNN	4	0.766 ± 0.015	0.791	116,864	0.12 μJ
E-SNN	8	0.766 ± 0.014	0.800	227,501	0.22 μJ
E-SNN	12	0.769 ± 0.022	0.789	339,801	0.32 μJ
Enc-ANN	4	0.787 ± 0.022	0.815	30,272	0.08 μJ
Enc-ANN	8	0.800 ± 0.015	0.820	29,881	0.06 μJ
Enc-ANN	12	0.787 ± 0.017	0.816	28,956	0.05 μJ
Matched ANN	–	0.756 ± 0.004	0.771	0	0.39 μJ

the remaining models in the same order as in Table II. Class-weighted training did not help either neural model; the E-SNN dropped to 0.753 ± 0.004 and the ANN to 0.747 ± 0.003.

B. Shorter time windows

The neural comparison is repeated in Table III with 4 and 8 time steps on the same partitions. For the E-SNN, macro-F1 at the 5% budget is 0.766 ± 0.015 at $T = 4$, 0.766 ± 0.014 at $T = 8$ and 0.769 ± 0.022 at $T = 12$. The three are indistinguishable given the seed spread. At the stricter 1% budget the shorter windows are, if anything, slightly better (0.791 and 0.800 against 0.789). The cost, however, falls almost in proportion to T: hidden-layer sparsity stays between 72% and 75%, so synaptic events drop from 339,801 to 116,864 per flow. The spike-input control is again the strongest neural model at every window, reaching 0.800 ± 0.015 at $T = 8$.

C. Architecture screen

Five E-SNN variants are listed in Table IV. The rule fixed before the run was to rank them by tuning-partition macro-F1 and confirm only the winner. The wider network ranked

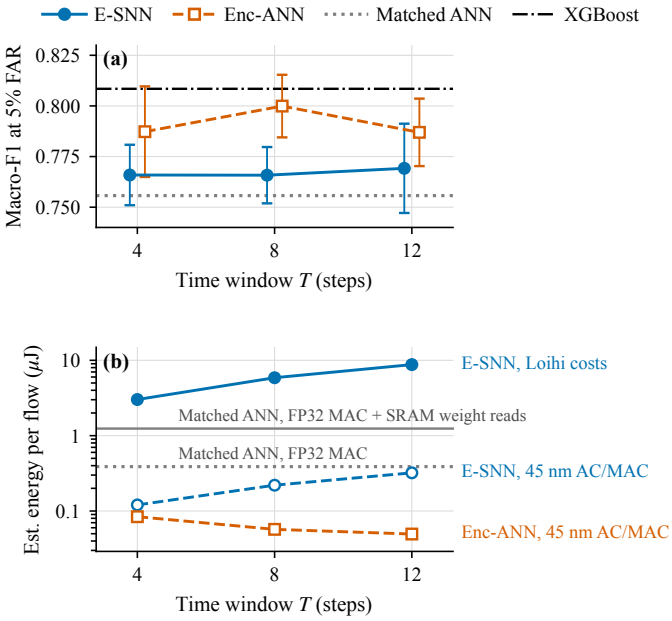


Fig. 2. Effect of the time window T. (a) Test macro-F1 at the 5% false-alarm budget (mean and s.d. over three seeds), with the matched ANN and XGBoost as references. (b) Estimated energy per flow on a log scale under two cost models: optimistic 45 nm accumulate and multiply-accumulate costs, and published Loihi per-operation costs for the E-SNN. The matched ANN is shown with and without SRAM weight reads.

first on tuning data by a clear margin, yet on test it was no better than the others, and every variant fell within the seed spread of the original. We read this as a sign that the tuning partition rewards fitting the easy majority classes, while the hard classes are limited by the label conflicts described in the next subsection. Retrained with all three seeds, the wide variant reaches 0.770 ± 0.014 at the 5% budget and 0.803 ± 0.007 at 1%. That is no better than the original at $T = 8$ (0.766 ± 0.014 and 0.800 ± 0.001), with 2.5 times

TABLE IV
ARCHITECTURE SCREEN AT $T = 8$, SEED 42. VARIANTS RANKED ON TUNING MACRO-F1 (ARGMAX DECISION); TEST MACRO-F1 AT THE 5% BUDGET. SEED 42 OF THE ORIGINAL REACHED 0.756 AT $T = 8$.

Variant (change from original)	Tune MF1	Test MF1
Wider layers (256, 128)	0.833	0.762
Weighted-average fusion only	0.802	0.760
Learning rate 10^{-3}, dropout 0.1	0.792	0.754
Eight receptive fields per feature	0.787	0.768
Population branch only	0.775	0.753

TABLE V
PER-CLASS TEST F1 AT THE 5% BUDGET, MEAN OF THREE SEEDS. CLASSES WITH F1 $\geq$ 0.96 FOR ALL MODELS ARE OMITTED (DOS-SLOWLORIS, FOR EXAMPLE, RANGES FROM 0.964 FOR THE E-SNN TO 0.998 FOR XGBOOST).

Class	E-SNN	ANN	Enc-ANN	RF	XGB
DoS-SlowHTTPTest	0.552	0.519	0.625	0.747	0.734
FTP-BruteForce	0.243	0.271	0.274	0.234	0.337
Infiltration	0.127	0.124	0.152	0.169	0.171
Web Attacks	0.360	0.171	0.393	0.319	0.464

TABLE VI
SYNAPTIC OPERATIONS PER FLOW AND A FIRST-ORDER ENERGY ESTIMATE. CPU TIME IS SINGLE-THREAD, BATCH 4,400.

Model	ACs	MACs	Est. energy	CPU time
E-SNN	339,801	3,328	0.32 μJ	57.1 μs
Matched ANN	0	84,420	0.39 μJ	1.4 μs
Enc-ANN	28,956	5,094	0.05 μJ	16.3 μs

TABLE VII
ENERGY PER FLOW WITH PUBLISHED LOIHI PER-OPERATION COSTS [3] FOR THE E-SNN (SYNAPTIC EVENTS AT 23.6 PJ, NEURON UPDATES AT 81 PJ, LAYER NORMALISATION AND FUSION ON A 45 NM HOST) AND A 45 NM DIGITAL ACCELERATOR FOR THE MATCHED ANN [12]. ESTIMATES, NOT MEASUREMENTS.

Model	Syn. events	Neuron upd.	Energy
E-SNN, $T = 4$	116,864	2,304	3.0 μJ
E-SNN, $T = 8$	227,501	4,608	5.9 μJ
E-SNN, $T = 12$	339,801	6,912	8.8 μJ
ANN (FP32)	–	–	0.40 μJ
ANN (FP32 + SRAM)	–	–	1.24 μJ

as many parameters (217,725). Its seed-42 run reproduced the screening result to the last digit (0.7620), which also confirms that training on Kaggle is deterministic for a fixed seed.

D. Where the errors are

Table V lists per-class F1. Nine of the thirteen classes are detected almost perfectly by every model. The differences between models come almost entirely from four classes: DoS-SlowHTTPTest, FTP-BruteForce, Infiltration and Web Attacks.

The feature-group audit explains three of these. Of 193,360 FTP-BruteForce flows, 193,353 fall into just 54 distinct vectors, and every one of those vectors also appears under another label. The reverse holds for DoS-SlowHTTPTest, where 139,884 of 139,890 flows share vectors with other classes. Both attacks leave short, nearly empty TCP connections that the flow extractor can summarise identically, and in that case no classifier working on these features can tell them apart. Infiltration is different. Its flows are spread over 140,610 groups, but 88,810 of them share a vector with benign traffic, which is consistent with the known labelling problems in this class [5]. Across the whole dataset, 68,467 groups carry conflicting labels, and a classifier that predicts the majority label of each group would still be wrong on 1.10% of rows. Any reported error below that floor on this representation is a sign of leakage.

Web Attacks behaves differently again. Its ranking quality is high (one-vs-rest ROC AUC 0.994 for the seed-42 E-SNN and 1.000 for XGBoost), yet its F1 is low because the class has only 179 test flows against 2.7 million benign ones. At the 5% budget the E-SNN finds 98 of the 179 attacks but also assigns 661 other flows to the class (precision 0.13). XGBoost finds 160 with 342 false assignments (precision 0.32). A small false-positive rate on the benign majority is enough to swamp a class this rare, so AUC overstates how usable the detector is for it, and average precision (0.49 and 0.85) gives a fairer picture.

E. Operation counts and runtime

We counted operations with forward hooks on 4,400 class-balanced flows. In a linear layer that receives binary input, each nonzero input contributes one accumulate (AC) per output neuron; any other linear layer contributes multiply-accumulates (MACs). Table VI reports counts per flow for the original $T = 12$ models. The energy column applies the 45 nm estimates of Horowitz [12] (0.9 pJ per 32-bit AC, 4.6 pJ per MAC) and should be read only as a comparison between the models. It ignores membrane updates, memory traffic and encoding, which often dominate on real hardware.

At $T = 12$ the E-SNN's hidden neurons are silent 72% of the time, yet with 12 time steps and three branches it still performs four times as many synaptic events as the matched ANN performs MACs. The cheaper accumulate makes up for this, and the estimate ends about 17% below the ANN. The spike-input control is far cheaper than both, because it sees each binary input once instead of propagating it through recurrent membrane state at every time step. On a general-purpose CPU, where sparsity brings no saving, the E-SNN is about 40 times slower than the ANN. Even so, the service scores more than 15,000 flows per second on a single laptop CPU, which is enough for a small network segment.

Table VII replaces the arithmetic-only figures with an estimate for neuromorphic hardware. We charge each synaptic event (a presynaptic spike times its fan-out) 23.6 pJ and each neuron update 81 pJ, the minimum synaptic-operation and active neuron-update energies reported for Intel's Loihi chip [3]. Layer normalisation and the fusion network have no native Loihi counterpart and are charged to a 45 nm host at 4.6 pJ per MAC [12]. Under these assumptions the E-SNN

TABLE VIII
FROZEN 2018 MODELS ON CIC-IDS2017. "RECAL.": THRESHOLD REFITTED ON HALF OF THE 2017 BENIGN FLOWS (5% BUDGET), ATTACK LABELS UNUSED. DR: SHARE OF ATTACK FLOWS THAT RAISE AN ALARM.

	2018 threshold		Recal.	
Model	FAR	DR	FAR	DR
E-SNN ($T = 12$)	0.262	0.630	0.050	0.230
E-SNN ($T = 4$)	0.228	0.420	0.050	0.239
XGBoost	0.252	0.494	0.050	0.317

needs about 3.0, 5.9 and 8.8 μJ per flow at $T = 4$, 8 and 12, against 0.40 μJ for the matched ANN on a 45 nm accelerator, or 1.2 μJ if every weight is also read from on-chip SRAM. Synaptic events make up more than 90% of the E-SNN total, and about three quarters of them come from the binary input encoding. The ANN is therefore cheaper by a factor of 2.4 to 22. This is an estimate built from published per-operation figures, not a measurement, and it ignores static power, routing between cores and the cost of generating the input spikes. The 45 nm accumulate figure used above is thus about 26 times more favourable to the SNN than the published cost of a synaptic operation on neuromorphic silicon, and the sign of the comparison depends on which one is used.

F. Transfer to a different network

A detector is only useful if it works outside the capture it was trained on. We therefore ran the frozen seed-42 models, unchanged, on the CIC-IDS2017 MachineLearningCVE export (2,214,468 labelled flows). That dataset was recorded a year earlier on a different testbed with an older version of the same flow extractor. The export we used lacks two of the original files (Tuesday, with the FTP and SSH brute-force attacks, and Thursday morning, with the web attacks), so those classes are absent. Its features match ours one to one, except that the protocol field is missing; we impute it from the destination port (UDP for common UDP services, TCP otherwise). Attack labels are mapped to the 2018 families; PortScan and Heartbleed have no 2018 counterpart.

The result is a clear negative, shown in Table VIII. With the 2018 threshold, the false-alarm rate rises from the calibrated 5% to between 23% and 26% for all three models, so the score distribution of normal traffic has shifted. We then gave each model the one adjustment an operator could make without attack labels: refitting the alarm threshold on half of the 2017 benign flows to a 5% budget, and evaluating on the other half and on all attacks. Detection then falls to 23%–32%. Only the slow application-layer DoS attacks transfer well. The 12-step E-SNN still flags 86% of GoldenEye and 87% of Slowloris flows, and the 4-step E-SNN flags 99.8% and 99.6%. Volumetric DDoS, DoS-Hulk and Bot traffic is largely missed by the SNNs, while XGBoost keeps 61% of Bot flows. No model detects the 36 Infiltration or 11 Heartbleed flows. Figure 3 breaks the recalibrated detection rate down by attack type.

To ask how much local labelled data an operator would need, we split the CIC-IDS2017 flows by feature-vector fingerprint into an adaptation pool, a calibration partition and a test partition (50/10/40), drew class-stratified labelled samples of 0.1%, 1% and 5% of the pool (1,101, 11,026 and 55,153 flows), and refitted the temperature and the 5% alarm threshold on the 2017 calibration partition. PortScan was added as a new class. Table IX gives means over three seeds. Labels recover detection quickly. With only 1,101 labelled flows, fine-tuning the 4-step E-SNN from its 2018 checkpoint raises detection from 27% to 98.5% at a 5.3% false-alarm rate, and XGBoost trained from scratch on the same flows reaches 99.6% at 5.0%. The fine-tuned E-SNN matches XGBoost on the DoS, DDoS and PortScan traffic but not on Bot flows, which it misses with 0.1% and 1% of the pool and detects at 32% with 5%, while XGBoost detects 97% to 99.5% with 1% or more. Continuing the 2018 XGBoost model cannot learn PortScan and stays at 72% to 75% detection. Because pool and test come from the same capture days, these figures are an optimistic bound for a real deployment. Macro-F1 stays low for every method because a 5% false-alarm rate on 671,643 benign test flows produces about 33,000 false alarms, which swamp the precision of the small classes.

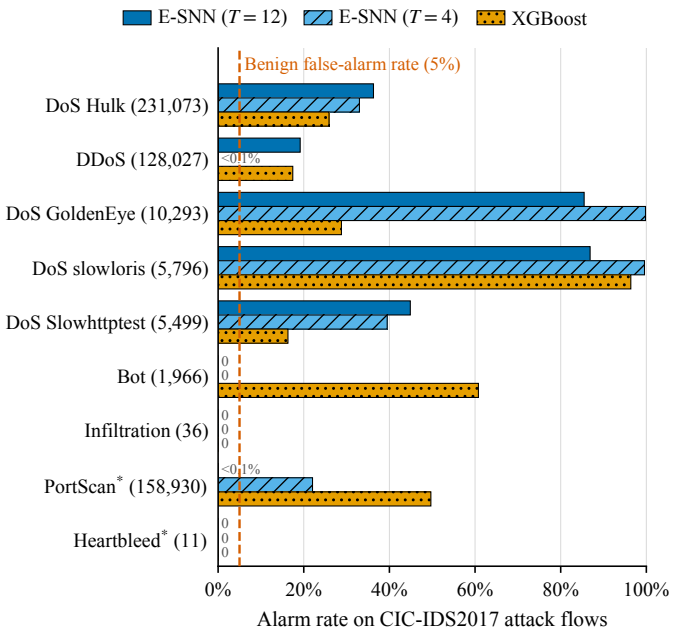


Fig. 3. Share of CIC-IDS2017 attack flows that raise an alarm after the threshold of each frozen 2018 model is refitted to a 5% benign false-alarm rate (dashed line). Flow counts are in parentheses; an asterisk marks attack types with no CSE-CIC-IDS2018 counterpart.

We see this as the most practically relevant finding of the study. High in-distribution scores on CSE-CIC-IDS2018, ours included, say little about how a flow-feature detector will behave on another network. Any deployment would need training or at least fine-tuning on local traffic, which, as Table IX shows, can be cheap in labels. Claims of generalisation should be backed by a cross-dataset test of this kind.

TABLE IX
LOCAL ADAPTATION ON CIC-IDS2017 (TEST PARTITION, MEAN OVER THREE SEEDS). BUDGET: SHARE OF THE 2017 ADAPTATION POOL WITH LABELS. ALL ROWS USE A 5% BENIGN ALARM THRESHOLD REFITTED ON 2017 CALIBRATION DATA.

Method	Budget	FAR	DR	Macro-F1
E-SNN T=4, frozen	0	0.062	0.268	0.188
XGBoost, frozen	0	0.050	0.314	0.291
E-SNN T=4, fine-tuned	0.1%	0.053	0.985	0.564
XGBoost, from scratch	0.1%	0.050	0.996	0.526
E-SNN T=4, fine-tuned	1%	0.059	0.994	0.592
XGBoost, from scratch	1%	0.050	1.000	0.665
E-SNN T=4, fine-tuned	5%	0.053	0.996	0.707
XGBoost, from scratch	5%	0.051	1.000	0.686
XGBoost, continued	5%	0.057	0.749	0.483

VI. A CONTAINERISED DETECTOR

To let others test the frozen models on their own captures, we package the seed-42 E-SNN checkpoints for $T = 12$ and $T = 4$, the XGBoost checkpoint, the fitted preprocessor and the calibration records in a Docker image. A FastAPI service accepts CICFlowMeter CSV files or JSON batches of flows and returns a class, an alarm flag and the attack log-odds for each flow, using the same 5% threshold as Table II. Before loading, the service checks each checkpoint's SHA-256 digest against its calibration record. A replay script streams flows to the service in small batches to imitate a live sensor. On a class-balanced sample of 4,400 flows from four capture days both E-SNN checkpoints raised alarms on every attack flow, and on 1.3% ($T = 12$) and 2.0% ($T = 4$) of benign flows. That sample is drawn from the public data and may overlap the test partition, so it shows that the service works but does not count as a further evaluation.

VII. DISCUSSION AND LIMITATIONS

What the spikes buy. Within this protocol the useful part of the SNN appears to be its input encoding. The population and latency codes give a plain feed-forward network a better view of skewed flow features, and that network then beats the spiking version on both accuracy and estimated energy. Short windows help the SNN's cost a great deal but do not close the accuracy gap, and the control profits from the same encoders at every window length. This does not rule out SNN advantages on neuromorphic hardware with on-chip encoding or on temporal inputs such as packet sequences. It does suggest that SNN papers on flow features should include a spike-input control before crediting membrane dynamics.

Energy needs a hardware model. The two cost models in Section V disagree about which network is cheaper. Counting accumulates at the price of a 45 nm adder favours the SNN; charging the published Loihi cost of a synaptic event, which includes the synapse memory access, favours the dense perceptron by a wide margin, mainly because three quarters of the SNN's events come from the binary input encoding with a fan-out of 128. An SNN for flow features would need a much sparser input code, or hardware with cheaper synaptic events, before an energy advantage is plausible. We think energy claims in this area should state their cost model and, ideally, report both.

Trees remain strong. Untuned XGBoost was the best model at every budget, in line with broader evidence on tabular data [8]. A neuromorphic detector would need to be judged against it, not only against other neural networks.

Limitations. This protocol (v3) was designed after we had seen test results of an earlier version, and the time-window and architecture experiments were run after the v3 test results were known. All of these are development results and not an independent confirmation, and the more configurations one tries on a single test partition, the more the best of them will flatter itself; we therefore report every configuration we ran. A fresh held-out dataset is still needed. Feature groups prevent verbatim duplication but not near-duplicates or temporal leakage within an attack episode, and the timestamps of the dataset are unverified. We used three seeds and fixed tree hyperparameters, and screened only five SNN variants. Finally, all energy numbers are estimates from operation counts; we made no hardware measurements.

VIII. CONCLUSION

On CSE-CIC-IDS2018 under a leakage-controlled, budget-matched protocol, an ensemble spiking network detects 96% of attack flows at a 5% false-alarm rate. It beats a parameter-matched perceptron, and it keeps the same accuracy with 4 time steps instead of 12, which cuts its operation count by a factor of three. Whether that makes it cheaper than a conventional network depends on the hardware cost model, and with published Loihi figures it does not. It still loses to gradient-boosted trees, and also to a plain network that is fed the same spike encodings. Much of the remaining error comes from flows that the feature extractor maps to identical vectors under different labels. None of the models, the SNN included, carries its accuracy over to a different network without retraining. We hope the protocol, the audit and the released detector make future SNN results on this benchmark easier to compare and harder to overstate.

REPRODUCIBILITY

Code, configuration, per-run calibration and evaluation records, frozen checkpoints and the Docker detector are available at https://github.com/ozgurucar/ids2018-ensemble-snn. Training ran on a single Kaggle GPU; package versions are pinned in the repository.

REFERENCES

[1] I. Sharafaldin, A. H. Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proc. 4th Int. Conf. Information Systems Security and Privacy (ICISSP)*, 2018, pp. 108–116.

[2] W. Maass, "Networks of spiking neurons: The third generation of neural network models," *Neural Networks*, vol. 10, no. 9, pp. 1659–1671, 1997.

[3] M. Davies *et al.*, "Loihi: A neuromorphic manycore processor with on-chip learning," *IEEE Micro*, vol. 38, no. 1, pp. 82–99, 2018.

[4] G. Engelen, V. Rimmer, and W. Joosen, "Troubleshooting an intrusion detection dataset: the CICIDS2017 case study," in *IEEE Security and Privacy Workshops (SPW)*, 2021, pp. 7–12.

[5] L. Liu, G. Engelen, T. Lynar, D. Essam, and W. Joosen, "Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018," in *IEEE Conf. Communications and Network Security (CNS)*, 2022, pp. 254–262.

[6] S. Kaufman, S. Rosset, C. Perlich, and O. Stitelman, "Leakage in data mining: Formulation, detection, and avoidance," *ACM Trans. Knowledge Discovery from Data*, vol. 6, no. 4, pp. 15:1–15:21, 2012.

[7] D. Arp, E. Quiring, F. Pendlebury, A. Warnecke, F. Pierazzi, C. Wressnegger, L. Cavallaro, and K. Rieck, "Dos and don'ts of machine learning in computer security," in *31st USENIX Security Symposium*, 2022, pp. 3971–3988.

[8] L. Grinsztajn, E. Oyallon, and G. Varoquaux, "Why do tree-based models still outperform deep learning on typical tabular data?" in *Advances in Neural Information Processing Systems (Datasets and Benchmarks Track)*, 2022.

[9] R. Sommer and V. Paxson, "Outside the closed world: On using machine learning for network intrusion detection," in *IEEE Symposium on Security and Privacy*, 2010, pp. 305–316.

[10] E. O. Neftci, H. Mostafa, and F. Zenke, "Surrogate gradient learning in spiking neural networks," *IEEE Signal Processing Magazine*, vol. 36, no. 6, pp. 51–63, 2019.

[11] J. K. Eshraghian, M. Ward, E. O. Neftci, X. Wang, G. Lenz, G. Dwivedi, M. Bennamoun, D. S. Jeong, and W. D. Lu, "Training spiking neural networks using lessons from deep learning," *Proceedings of the IEEE*, vol. 111, no. 9, pp. 1016–1054, 2023.

[12] M. Horowitz, "Computing's energy problem (and what we can do about it)," in *IEEE Int. Solid-State Circuits Conf. (ISSCC)*, 2014, pp. 10–14.

[13] S. Zhou and X. Li, "Spiking neural networks with single-spike temporal-coded neurons for network intrusion detection," in *Proc. 25th Int. Conf. Pattern Recognition (ICPR)*, 2021, pp. 8148–8155, arXiv:2010.07803.

[14] Z. Wang, F. A. Ghaleb, A. Zainal, M. M. Siraj, and X. Lu, "An efficient intrusion detection model based on convolutional spiking neural network," *Scientific Reports*, vol. 14, p. 7054, 2024.

[15] M. Živadinović and D. Simić, "Resource efficient Internet-of-Things intrusion detection with spiking neural networks," in *Position Papers of the 19th Conf. Computer Science and Intelligence Systems (FedCSIS)*, ser. Annals of Computer Science and Information Systems, vol. 40, 2024, pp. 73–78.

[16] A. Fesu, N. Shone, Á. MacDermott, B. Zhou, and M. H. Eiza, "Comparative analysis of SNN and CNN models for energy efficient intrusion detection," in *Proc. 12th IFIP Int. Conf. New Technologies, Mobility and Security (NTMS)*, 2025, pp. 1–7.

[17] M. G. Karthik, V. Keerthika, S. V. Mantena, D. Siri, L. P. Yeluri, K. K. Lella, and B. R. Ganesh, "Energy-efficient intrusion detection with a protocol-aware transformer–spiking hybrid model," *Scientific Reports*, vol. 16, p. 7095, 2026.

[18] R. Patel, S. Mitra, D. Amebley, T. Akinrele, S. Dibbo, S. Mittal, and S. Rahimi, "The value of spike timing: A leakage-resistant benchmark of SNN design choices for network intrusion detection," in *IEEE Int. Conf. Cognitive Machine Intelligence (CogMI)*, 2026, to appear. arXiv:2606.01442.

[19] M. Z. Alom and T. M. Taha, "Network intrusion detection for cyber security on neuromorphic computing system," in *Proc. Int. Joint Conf. Neural Networks (IJCNN)*, 2017, pp. 3830–3837.

[20] A. Mehrabi and A. van Schaik, "Intrusion detection on industrial networks using neuromorphic computing," in *Proc. Int. Conf. Electrical, Control and Instrumentation Engineering (ICECIE)*, 2024.

[21] M. Saerens, P. Latinne, and C. Decaestecker, "Adjusting the outputs of a classifier to new a priori probabilities: A simple procedure," *Neural Computation*, vol. 14, no. 1, pp. 21–41, 2002.

[22] C. Guo, G. Pleiss, Y. Sun, and K. Q. Weinberger, "On calibration of modern neural networks," in *Proc. 34th Int. Conf. Machine Learning (ICML)*, 2017, pp. 1321–1330.

[23] X. Tong, Y. Feng, and J. J. Li, "Neyman-pearson classification algorithms and NP receiver operating characteristics," *Science Advances*, vol. 4, no. 2, p. eaao1659, 2018.

[24] L. Breiman, "Random forests," *Machine Learning*, vol. 45, no. 1, pp. 5–32, 2001.

[25] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proc. 22nd ACM SIGKDD Int. Conf. Knowledge Discovery and Data Mining*, 2016, pp. 785–794.

[26] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin, "CatBoost: Unbiased boosting with categorical features," in *Advances in Neural Information Processing Systems*, 2018.